1.Goal of CPU Scheduling

CPU scheduling tries to make the system **fast and efficient**:

1. **Max CPU Utilization** → CPU should not stay idle
2. **Max Throughput** → Complete more processes in less time
3. **Min Turnaround Time** → Finish each process quickly
4. **Min Waiting Time** → Processes should not wait too long
5. **Min Response Time** → Quick response when a process starts

👉 *Shortcut to remember:*
Max use & speed, Min delay

2.Convoy Effect & FCFS

First-Come, First-Served (FCFS) Scheduling – Theory

- FCFS is the simplest CPU scheduling algorithm.
- Processes are executed in the order they arrive in the ready queue.
- It is non-preemptive: once a process gets the CPU, it runs until completion or waiting.
- Easy to implement using a FIFO queue.
- Fair in the sense that every process is served in arrival order, but not always efficient.

Convoy effect is a problem in FCFS CPU scheduling where a long process occupies the CPU and all short processes have to wait behind it.

Convoy Effect = Long process first → short processes wait → overall performance becomes slow

Example

Process	Burst Time
P1	12 ms
P2	2 ms
P3	1 ms

👉 Order: **P1 → P2 → P3**

FCFS does not consider burst time; it runs in arrival order.
So this increases average waiting time and reduces system performance.

Effects

- Increased waiting time
- Low CPU utilization
- Poor throughput

Low system performance

3.Deadlock:

A deadlock is a situation where two or more processes are waiting for each other's resources, so none of them can proceed.

DEADLOCK Reason / Necessary Conditions:

1. **Mutual Exclusion:**
   Only one process can use a resource at a time.
2. **Hold and Wait:**
   A process holds one resource and waits for another.
3. **No Preemption:**
   Resources cannot be taken forcefully; they are released only by the process.
4. **Circular Wait:**
   Processes form a circular chain, each waiting for the next.

Example:

Assume two processes and two resources:

- **P1** holds **Printer** and waits for **Scanner**
- **P2** holds **Scanner** and waits for **Printer**

Situation:

- P1 → waiting for P2
- P2 → waiting for P1

Both are waiting for each other → **no one can proceed → Deadlock**

4.Deadlock Prevention

Deadlock prevention is a method to **avoid deadlock by eliminating at least one of its four necessary conditions**.

Methods:

1. **Mutual Exclusion Prevention**
   Try to make resources **shared** so many processes can use them at the same time.

2. **Hold and Wait Prevention**
   Do not allow a process to hold one resource and wait for others.
   ☞ A process must:

- Take all resources at once, **or**

- Take resource only when it has none

3. **No Preemption Prevention**
   If a process cannot get a resource, it must **release all resources it has**.
   ☞ It will get them again later.

4. **Circular Wait Prevention**
   Give numbers to resources (like R1, R2, R3).
   ☞ A process must request resources in order (R1 → R2 → R3).
   ⮕ So, no cycle can form.

Example:

If a process must take R1 before R2, it cannot create a loop → **no deadlock**

5.Base and Limit Registers

Base and Limit registers are used in operating systems to **protect memory and control access** of a process.

- The **base register** stores the starting address of a process.
- The **limit register** stores the size (range) of that process.

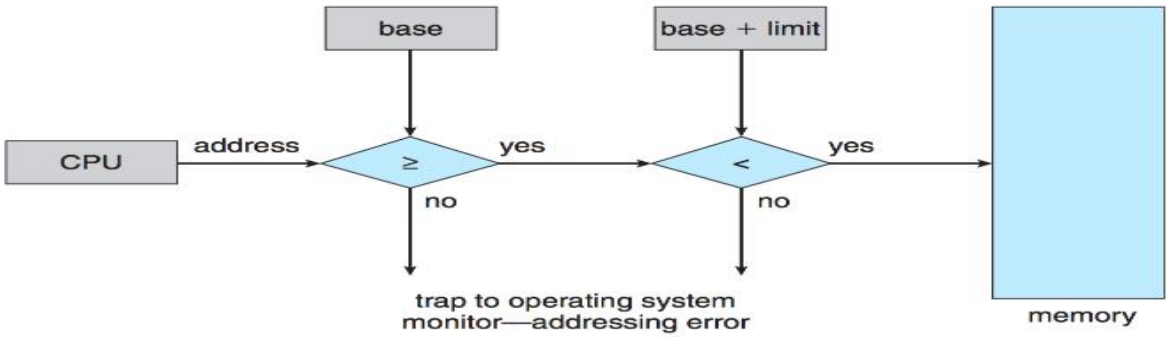


Figure 8.2 Hardware address protection with base and limit registers.

Concepts (Base & Limit Register)

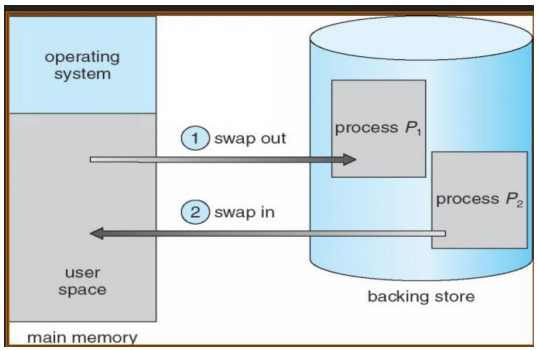
1. CPU creates a logical address.
2. The system checks if the address ≥ base and address < base + limit.
3. If valid, the logical address is added to the base register.
4. A physical address is generated and memory access is allowed.
5. Otherwise, an error (trap) occurse

6.Swapping

Swapping is a memory management technique where a process is **moved from main memory (RAM) to disk temporarily** and later brought back to RAM for execution.

- It helps manage limited memory efficiently.
- Used in systems like UNIX, Linux, and Windows.

- Used when RAM is full.
    - Some processes are moved to a **backing store (disk)**.



Concepts of Swapping

- A process is first loaded into **main memory (RAM)** for execution.
- When RAM becomes full, some process is **moved to disk (swap out)**.
- Another process is then **brought into RAM (swap in)**.
- The swapped-out process can later come back to RAM for execution when needed.

Classification of Security Violations

1. Breach of Confidentiality

👉 Means: **Secret data is seen by unauthorized person**
📌 Easy idea: *Data leak*

Example:

- Hacker reads private files
- Someone checks your personal messages

---

2. Breach of Integrity

👉 Means: **Data is changed without permission**
📌 Easy idea: *Data is modified*

Example:

- Changing marks in database
- Editing bank balance

---

3. Breach of Availability

👉 Means: **Data or system is destroyed or not accessible**
📌 Easy idea: *System/data not usable*

Example:

- Deleting important files
- System crash

---

4. Theft of Service

👉 Means: **Using someone else's resources without permission**
📌 Easy idea: *Free use of service illegally*

Example:

- Using another person's WiFi
- Using someone's account

---

6. **Denial of Service (DoS)**
7. **Means:** Blocking real users from using system
   Easy idea: ***System is overloaded***

Example:

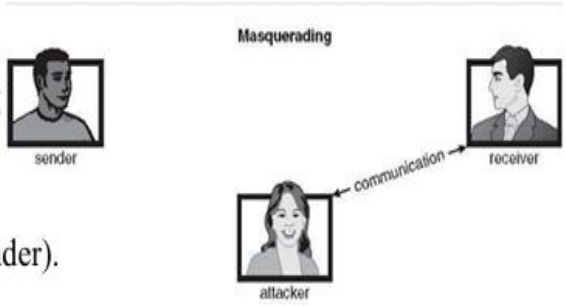
- Sending too many requests to server
- Website becomes slow or crashes

Quick Memory Trick 🧠

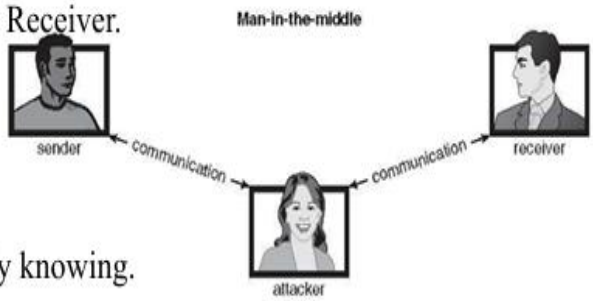
👉 **C I A T D**

- C → Confidentiality (see data)
- I → Integrity (change data)
- A → Availability (destroy/block data)
- T → Theft (use service)
- D → DoS (block users)

Security Violation Methods: Man-in-the-Middle & Masquerading



1. **Masquerading Attack**
   - An attacker pretends to be an authorized user.
   - They intercept communication by posing as the receiver (or sender).
   - Goal: gain access or escalate privileges.
   - **Illustration:** Sender → Attacker (pretending as Receiver) → Receiver.



2. **Man-in-the-Middle Attack**
   - Attacker secretly sits between sender and receiver.
   - They intercept, modify, or relay messages without either party knowing.
   - Both sender and receiver think they are talking directly, but attacker controls the flow.
   - **Illustration:** Sender → Attacker → Receiver (attacker relays both ways).